Artificial Intelligence

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 8 - PAO 2024 – II

Agenda

1. Redes Neuronales MLP
2. Aprendizaje en las redes MLP
3. MLP como clasificadores lineales y no lineales
4. Componentes de la función de error

Redes Neuronales

Entrenamiento

- Buscamos una estrategia iterativa que permita a la neurona adapte sus parámetros (pesos) de conexión, cuando se presentan los datos de entrenamiento en la entrada de la red
- El entrenamiento del perceptrón se basa en una ley de aprendizaje general, conocida como la regla delta:
  - Para cada peso W se realiza un ajuste basado en un ΔW, y de acuerdo con la siguiente regla:

$$\Delta W_{ij} = \alpha (T_j - Y_j) Y_i$$

Donde:

α es la velocidad de aprendizaje. Un factor entre $0 < \alpha \leq 1$

Y_i valor de salida de la neurona i (o la entrada a la neurona a partir de la segunda capa)

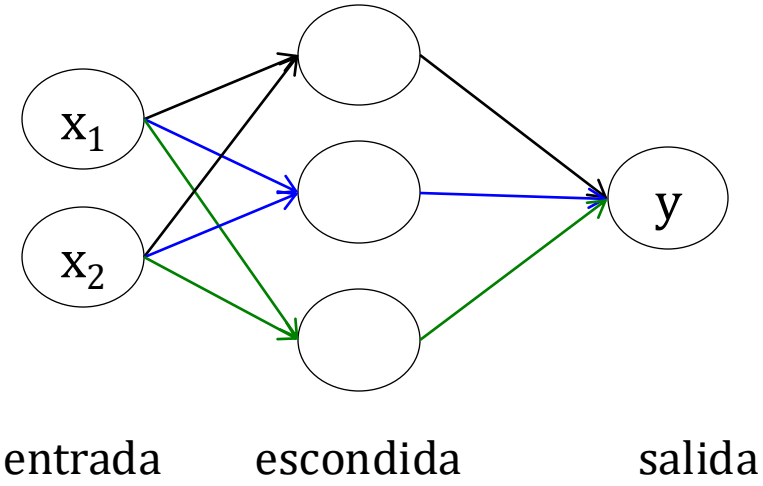
T_j valor deseado, esperado o target conocido en la neurona j

Y_j valor de salida calculado por la neurona j

Propagación hacia delante - forward propagation

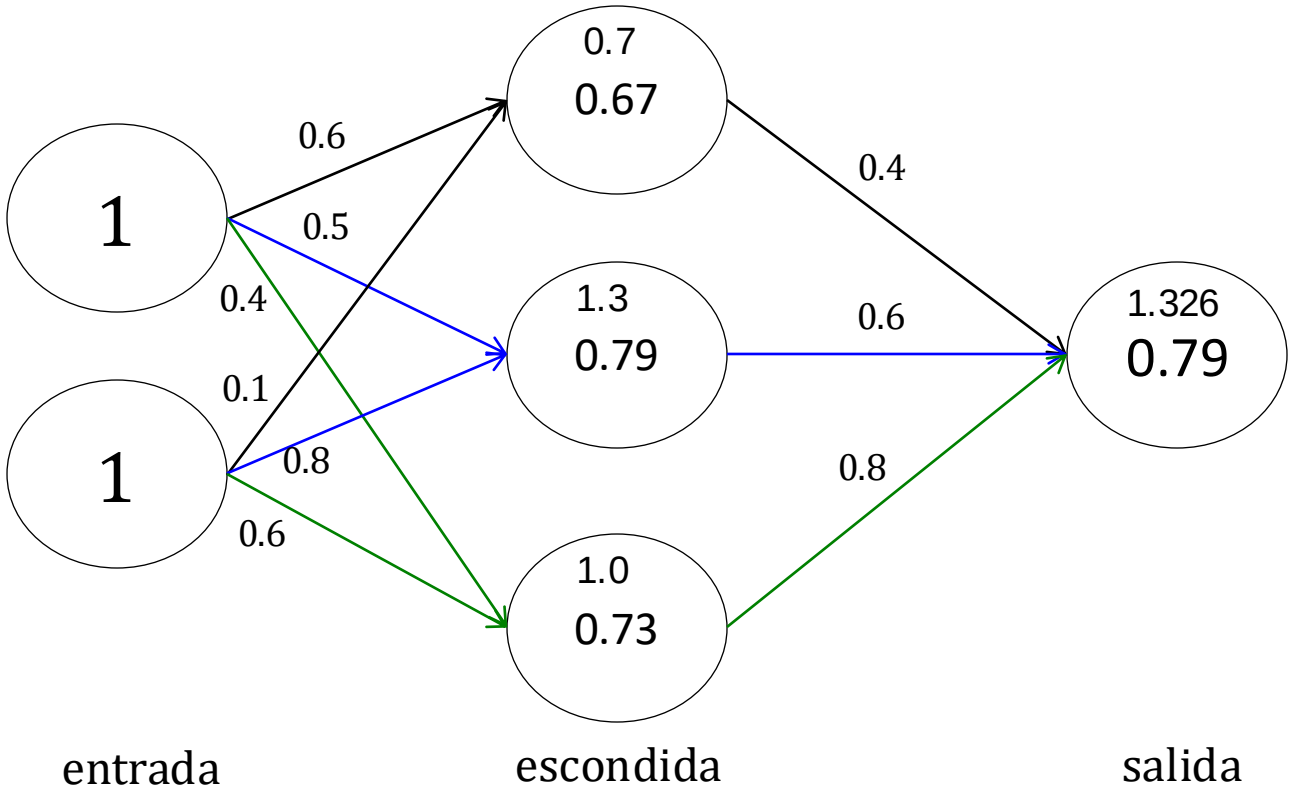
- ✓ Considere la siguiente red de 3 capas y 4 unidades de activación sigmoidal (neuronas con una función de umbral tipo sigmoidal)
- ✓ Entrenar la red y determinar la salida de la red utilizando los patrones de entrenamiento de la siguiente tabla

x_1 x_2	y
0, 0	0
0, 1	1
1, 0	1
1, 1	0



Para simplificar los cálculos, vamos a omitir el bias

Propagación hacia delante - forward propagation



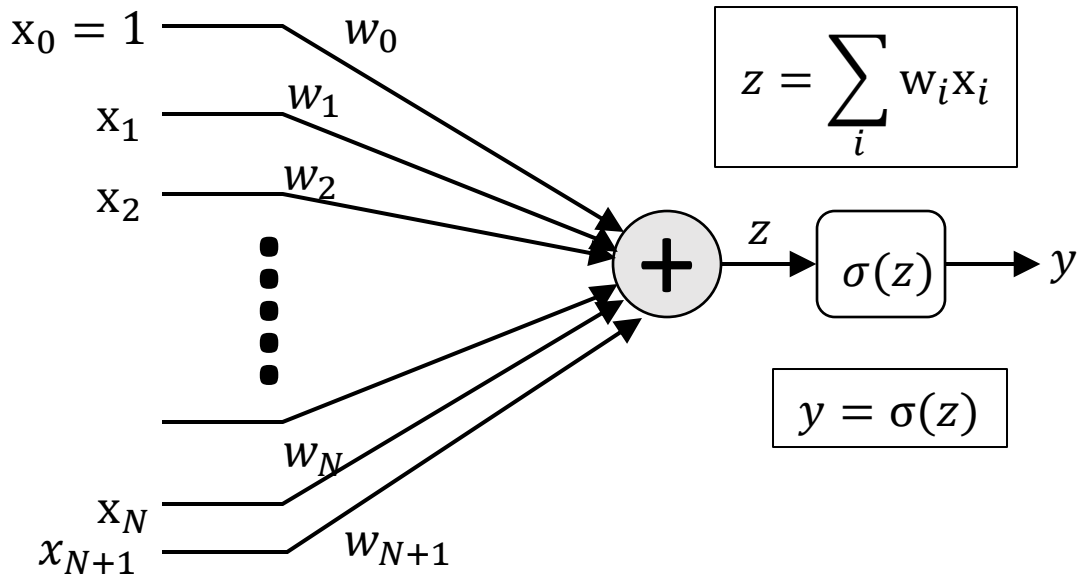
Valor esperado: 0
Calculado: 0.79

Propagación hacia delante - forward propagation

- ✓ Debido a que usamos un conjunto aleatorio de pesos iniciales, el valor total en la neurona de salida está errado: +0.79 (y el objetivo era 0)
- ✓ Si detenemos aquí el aprendizaje, este conjunto de pesos no servirá para reconocer los patrones de la entrada (operación XOR)
- ✓ Cómo calcular el error entre el valor esperado (0) y la salida real de la red (+0.79)?
- ✓ Luego de calcular el error, podemos aplicar la regla de aprendizaje Delta para aprender (corregir / descubrir) los pesos de la red, utilizando:

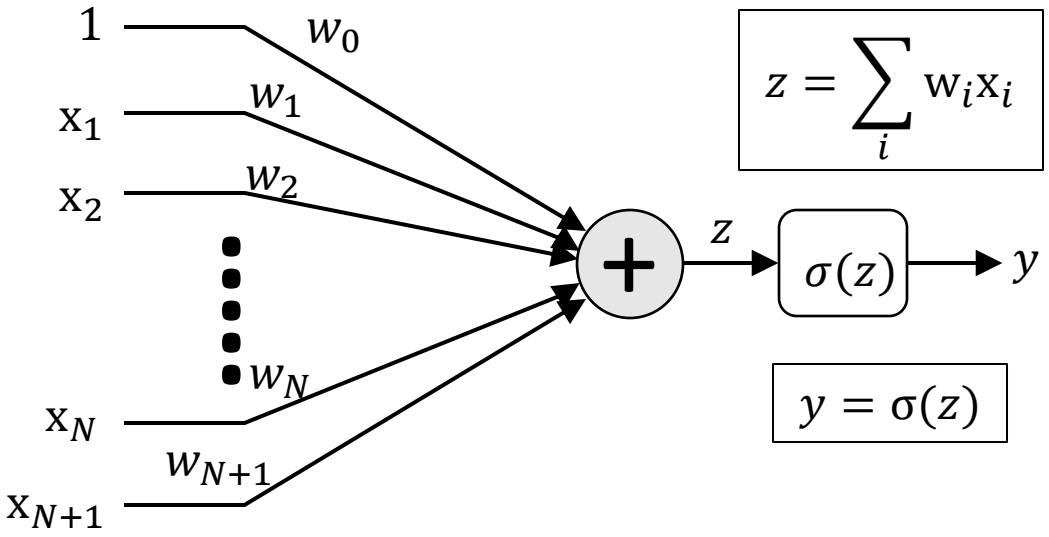
$$wnuevo = wanterior + \alpha(t - y)x$$

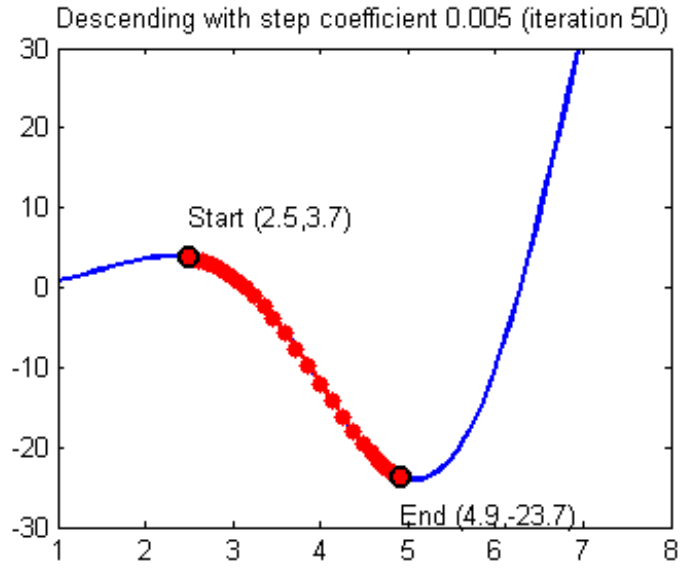
Aprendizaje



- El bias también se puede ver como el peso de otro componente de entrada que está fijo en 1
- Notar que: Si el bias no se menciona de manera explícita, siempre está. Recuerde que cada perceptron siempre tiene una entrada adicional (bias) que está fija en 1

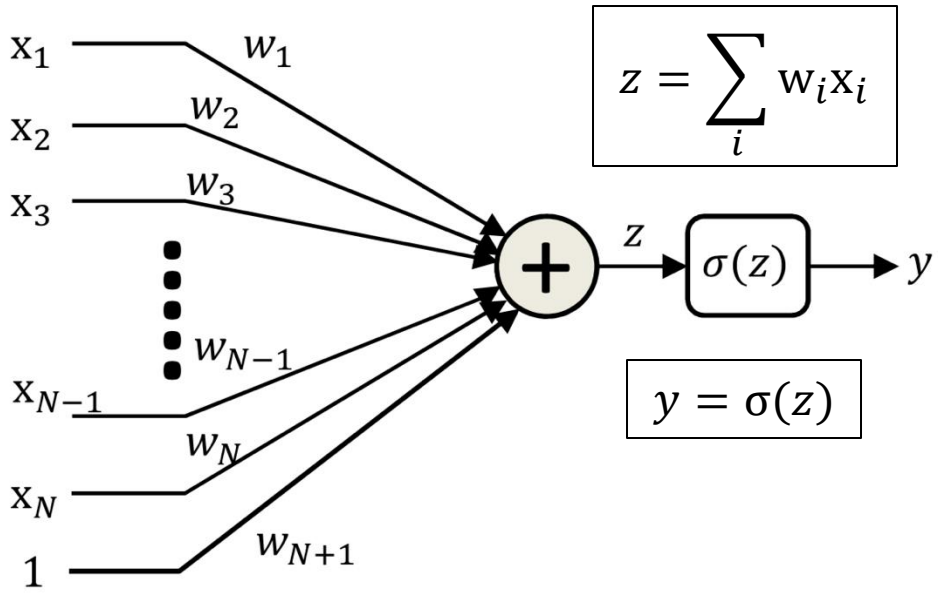
Cómo aprendemos la función $g(X)$?





- y es una función que depende de z; $\sigma(Z)$
- z es una función que depende de w_i y x_i
- Esto quiere decir que podemos calcular los cambios en la salida y, cuando ocurren pequeños cambios en la entrada o en los pesos
- El mecanismo que nos permite determinar esos cambios, de manera iterativa, es el gradiente descendiente !

Aprendizaje: Perceptron con activación derivable



$$z = \sum_i w_i x_i$$

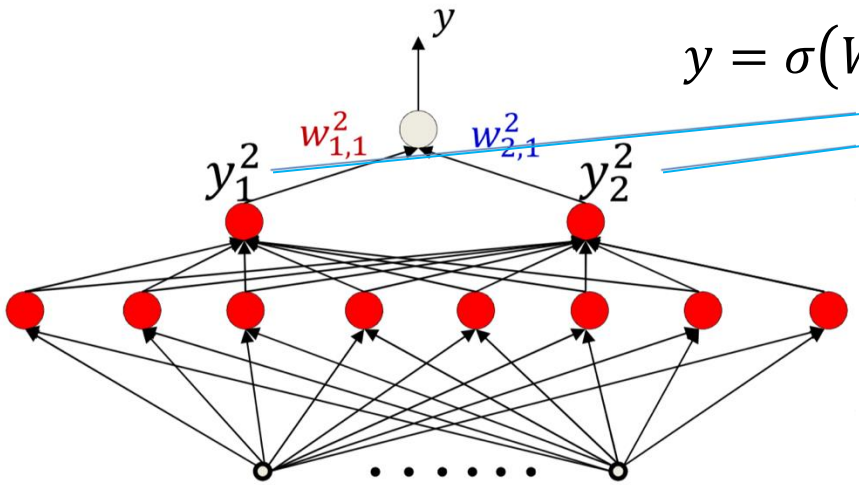
$$y = \sigma(z)$$

$$\frac{dy}{dw_i} = \frac{dy}{dz}\frac{dz}{dw_i}$$

$$\frac{dy}{dX_i} = \frac{dy}{dz}\frac{dz}{dX_i}$$

- Estas derivadas nos dicen cuan sensible es la función con respecto a las variables que la definen; es decir, cómo un cambio de la variable impacta al valor total de la función
- Durante el aprendizaje queremos determinar cómo cambia el error total, cuando se produce un cambio en los pesos

Aprendizaje: Red MLP derivable



$$y = \sigma\left(W_{1,1}^2 y_1^2 + W_{2,1}^2 y_2^2\right)$$

y = salida total de la red

$W_{i,j}^k$ = peso de la conexión de la unidad i^{th} de la capa k^{th} a la unidad j^{th} de la capa $(k+1)^{th}$

y_i^k = salida de la unidad i^{th} de la capa k^{th}

$\sigma(\,)$ = función de activación o umbral (derivable c.r.a w, x)

- Cada perceptron es derivable c.r.a sus pesos (incluyendo el peso del "bias") y entradas
- En Backprop se calcula la derivada de la función que define el error con respecto a cada peso w; así, determinamos cuán sensible es la función de error con respecto a cada uno de estos pesos
- Esto nos ayuda a determinar cuánto debemos cambiar cada parámetro w durante el descenso del gradiente; al final obtenemos el modelo que mejor se ajusta a los datos de entrenamiento

Aprendizaje: El problema

- Dado el conjunto de pares de datos de entrenamiento, de entrada-salida:

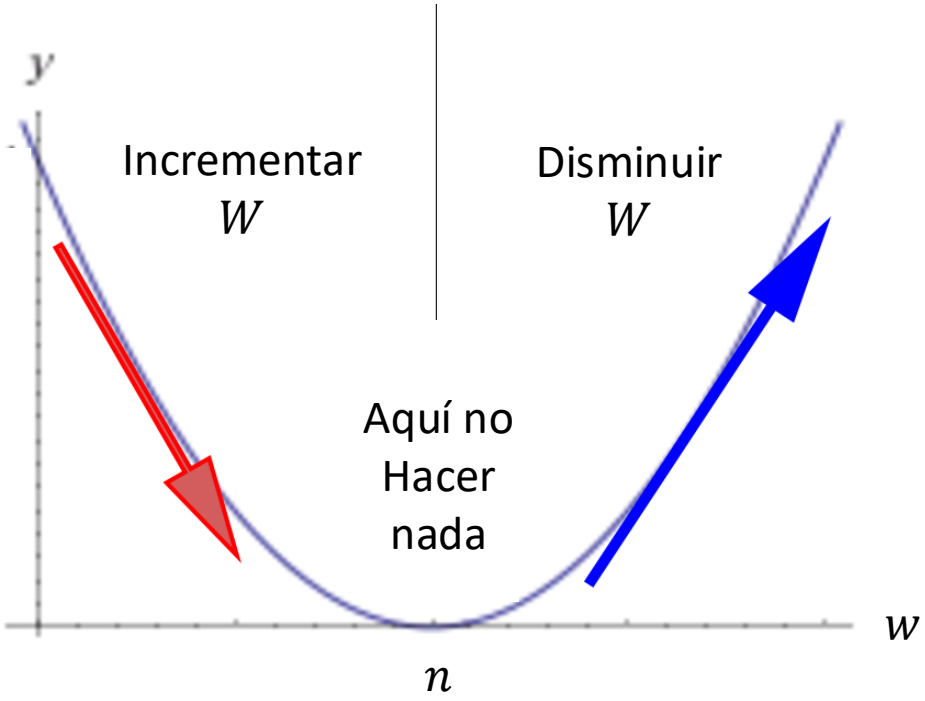
$$(X_1, d_1), (X_2, d_2), \ldots, (X_N, d_N)$$

- Minimizar la función:

$$Error(W) = \frac{1}{N} \sum_i div\,(f(X_i; W), d_i), \qquad \text{c.r.a } W$$

- Esto es un problema de minimización de funciones !!

Aprendizaje: Minimización de funciones



- Si $w < n$, la derivada es negativa, lo que significa que un incremento del peso, disminuye la función de error.
- En $w = n$, el error es 0, la derivada es 0, es un modelo perfecto, no se necesita cambiar nada
- Si $w > n$, el error vuelve a ser positivo, pero la derivada también es positiva, lo que significa que cualquier aumento en el peso aumentará aún más el error